



MANIPAL

ACADEMY of HIGHER EDUCATION

(Institution of Eminence Deemed to be University)

MANIPAL SCHOOL OF INFORMATION SCIENCES

(A Constituent unit of MAHE, Manipal)

LOS THEORY– Assignment 5

Reg. Number	Name	Branch
261100690024	K P NIDHI	CyberSecurity



MANIPAL SCHOOL OF INFORMATION SCIENCES

MANIPAL

(A constituent unit of MAHE, Manipal)

Command-Line Arguments, File Operators, and Filters

1. Theoretical Background

Before writing the scripts, here's a quick recap of the main pieces each one uses:

Positional Parameters

\$1, \$2, etc. are the arguments passed in when you run a script. \$0 is the script's own name, and \$# tells you how many arguments were given. I used \$# -eq 0 in every script to check whether an argument was actually passed before trying to use it.

read and Filters

The read command is what lets a script pause and take input from whoever is running it. For filtering log files, head grabs the first N lines, grep searches for a pattern, and piping grep into sort, uniq -c, and sort -nr (like in the earlier sort lab) lets you count and rank matches.

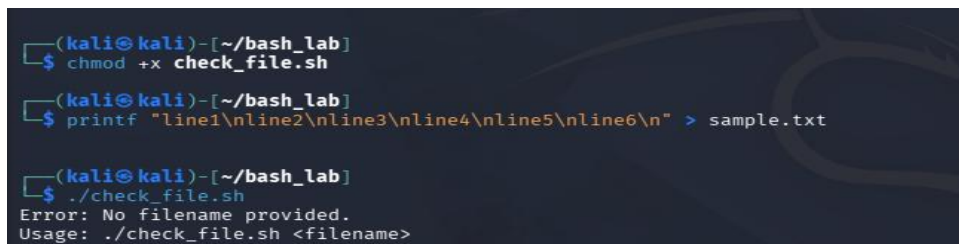
2. Basic File Existence Check — *check_file.sh*

This script takes a filename as the first argument. It first checks that an argument was actually given, then checks if the file exists using -e, and if it does, prints the first 5 lines with head.

```
#!/bin/bash
if [ $# -eq 0 ]; then
    echo "Error: No filename provided."
    echo "Usage: $0 <filename>"
    exit 1
fi

filename="$1"

if [ -e "$filename" ]; then
    echo "First 5 lines of $filename:"
    head -n 5 "$filename"
else
    echo "Error: File '$filename' does not exist."
    exit 1
fi
```



```
(kali@kali)~[/bash_lab]
$ chmod +x check_file.sh

(kali@kali)~[/bash_lab]
$ printf "line1\nline2\nline3\nline4\nline5\nline6\n" > sample.txt

(kali@kali)~[/bash_lab]
$ ./check_file.sh
Error: No filename provided.
Usage: ./check_file.sh <filename>
```

Figure 1: Making the script executable, creating sample.txt, and running check_file.sh with no argument.

Observation / Explanation: Since I didn't pass any argument, \$# is 0, so it prints the usage message and stops. It never even gets to check the file.

```
(kali@kali)~[/bash_lab]
$ ./check_file.sh sample.txt
First 5 lines of sample.txt:
line1
line2
line3
line4
line5
```

Figure 2: Running check_file.sh against an existing file.

Observation / Explanation: sample.txt exists, so the -e check passes and head -n 5 prints the first 5 of its 6 lines, like it's supposed to.

```
(kali@kali)~[/bash_lab]
$ ./check_file.sh missing.txt
Error: File 'missing.txt' does not exist.
```

Figure 3: Running check_file.sh against a file that does not exist.

Observation / Explanation: missing.txt doesn't exist, so -e fails and it goes into the else block and prints the error message instead of crashing on head.

3. File Type and Permission Checker — filetype_checker.sh

This one takes a path and figures out if it's a regular file or a directory, then checks the read, write, and execute permissions separately using -f, -d, -r, -w, and -x.

```
#!/bin/bash
if [ $# -eq 0 ]; then
    echo "Usage: $0 <path>"
    exit 1
fi

path="$1"

if [ ! -e "$path" ]; then
    echo "Error: '$path' does not exist."
    exit 1
fi

if [ -f "$path" ]; then
    echo "$path is a regular file."
elif [ -d "$path" ]; then
    echo "$path is a directory."
else
    echo "$path is neither a regular file nor a directory."
fi

[ -r "$path" ] && echo "$path is readable." || echo "$path is NOT readable."
[ -w "$path" ] && echo "$path is writable." || echo "$path is NOT writable."
[ -x "$path" ] && echo "$path is executable." || echo "$path is NOT executable."
```

```
(kali㉿kali)-[~/bash_lab]
$ chmod +x filetype_checker.sh

(kali㉿kali)-[~/bash_lab]
$ ./filetype_checker.sh sample.txt
sample.txt is a regular file.
sample.txt is readable.
sample.txt is writable.
sample.txt is NOT executable.
```

Figure 4: filetype_checker.sh run against the regular file sample.txt.

Observation / Explanation: -f picks up sample.txt as a regular file. It shows readable and writable since those bits are set, but not executable because a plain .txt file doesn't have the execute bit on.

```
(kali㉿kali)-[~/bash_lab]
$ ./filetype_checker.sh .
. is a directory.
. is readable.
. is writable.
. is executable.
```

Figure 5: filetype_checker.sh run against '.', the current directory.

Observation / Explanation: -d picks up '.' as a directory. Directories usually have the execute bit set for the owner (you need that just to cd into them), so all three checks come back true here.

```
(kali㉿kali)-[~/bash_lab]
$ chmod -r sample.txt 2>/dev/null; ./filetype_checker.sh sample.txt
sample.txt is a regular file.
sample.txt is NOT readable.
sample.txt is writable.
sample.txt is NOT executable.
```

Figure 6: Removing the read bit with chmod -r and re-running the checker.

Observation / Explanation: After removing the read bit with chmod -r, the -r check now says NOT readable, which matches the permission change. Note this is just checking the permission bit itself if I were logged in as root I could probably still open the file anyway, but -r doesn't care about that, it only looks at the bit.

```
(kali㉿kali)-[~/bash_lab]
$ chmod 644 sample.txt
```

Figure 7: Restoring normal permissions with chmod 644 before continuing.

4. Log Keyword Search — log_search.sh

This script takes a log file and searches it for four keywords failed, denied, unauthorized, and error and prints the matching lines under each keyword's own heading so it's easy to scan.

```
#!/bin/bash
if [ $# -eq 0 ]; then
    echo "Usage: $0 <logfile>"
    exit 1
fi

logfile="$1"
```

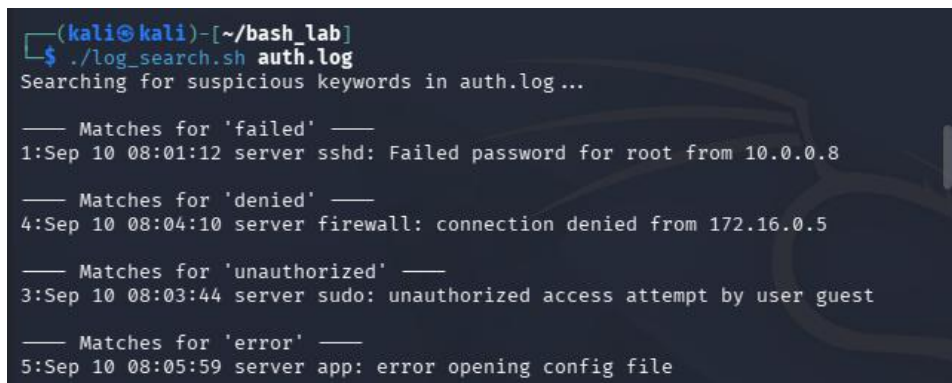
```

if [ ! -f "$logfile" ]; then
    echo "Error: '$logfile' is not a valid file."
    exit 1
fi

echo "Searching for suspicious keywords in $logfile..."
echo

for keyword in failed denied unauthorized error; do
    echo "---- Matches for '$keyword' ----"
    grep -in "$keyword" "$logfile"
    echo
done

```



```

(kali@kali)-[~/bash_lab]
$ ./log_search.sh auth.log
Searching for suspicious keywords in auth.log...

---- Matches for 'failed' ----
1:Sep 10 08:01:12 server sshd: Failed password for root from 10.0.0.8

---- Matches for 'denied' ----
4:Sep 10 08:04:10 server firewall: connection denied from 172.16.0.5

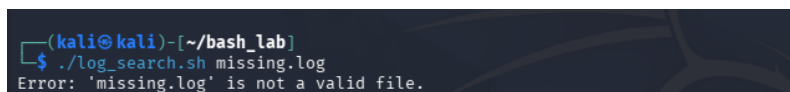
---- Matches for 'unauthorized' ----
3:Sep 10 08:03:44 server sudo: unauthorized access attempt by user guest

---- Matches for 'error' ----
5:Sep 10 08:05:59 server app: error opening config file

```

Figure 8: log_search.sh scanning auth.log for all four keywords.

Observation / Explanation: grep -in does a case-insensitive search (-i) and also shows the line number (-n). I set up my test log so each line only matches one keyword, so the output cleanly splits into the failed password attempt, the denied connection, the unauthorized access try, and the config error, each under its own heading.



```

(kali@kali)-[~/bash_lab]
$ ./log_search.sh missing.log
Error: 'missing.log' is not a valid file.

```

Figure 9: log_search.sh run against a log file that does not exist.

Observation / Explanation: missing.log isn't a real file, so -f fails and it errors out before it even tries to grep anything.

5. Empty vs Non-Empty File Check — check_empty.sh

This script checks that the file exists, that it's a regular file (not a directory), and then uses -s to check if it's empty or not. It only runs head if there's actually something in the file.

```

#!/bin/bash
if [ $# -eq 0 ]; then
    echo "Usage: $0 <filename>"
    exit 1
fi

filename="$1"

```

```

if [ ! -e "$filename" ]; then
    echo "Error: File '$filename' does not exist."
    exit 1
fi

if [ ! -f "$filename" ]; then
    echo "Error: '$filename' is not a regular file."
    exit 1
fi

if [ -s "$filename" ]; then
    echo "'$filename' is non-empty. First 5 lines:"
    head -n 5 "$filename"
else
    echo "'$filename' exists but is empty."
fi

```



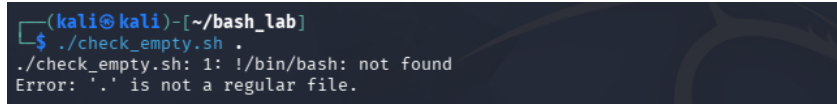
```

(kali@kali)-[~/bash_lab]
$ ./check_empty.sh sample.txt
./check_empty.sh: 1: !/bin/bash: not found
'sample.txt' is non-empty. First 5 lines:
line1
line2
line3
line4
line5

```

Figure 10: `check_empty.sh` run against the non-empty `sample.txt`.

Observation / Explanation: `sample.txt` has content, so `-s` is true and it prints the first 5 lines, as expected.



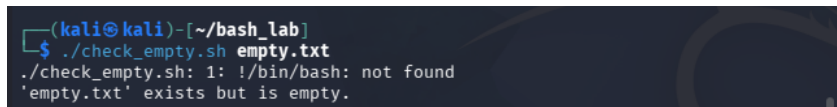
```

(kali@kali)-[~/bash_lab]
$ ./check_empty.sh .
./check_empty.sh: 1: !/bin/bash: not found
Error: '.' is not a regular file.

```

Figure 11: `check_empty.sh` run against `'.'`, a directory rather than a regular file.

Observation / Explanation: `'.'` exists, but it's a directory, not a regular file, so the `-f` check stops it right there before it even gets to check if it's empty.



```

(kali@kali)-[~/bash_lab]
$ ./check_empty.sh empty.txt
./check_empty.sh: 1: !/bin/bash: not found
'empty.txt' exists but is empty.

```

Figure 12: `check_empty.sh` run against an empty file.

Observation / Explanation: `touch empty.txt` makes a 0-byte file, so `-s` comes back false and it says the file is empty instead of trying to head an empty file.

6. Interactive Investigation — *investigate.sh*

This last script checks that the log file exists and is readable, then asks me to type in a search term. It shows the first 5 matching lines and then tells me how many matches there were in total.

```
#!/bin/bash
if [ $# -eq 0 ]; then
    echo "Usage: $0 <logfile>"
    exit 1
fi

logfile="$1"

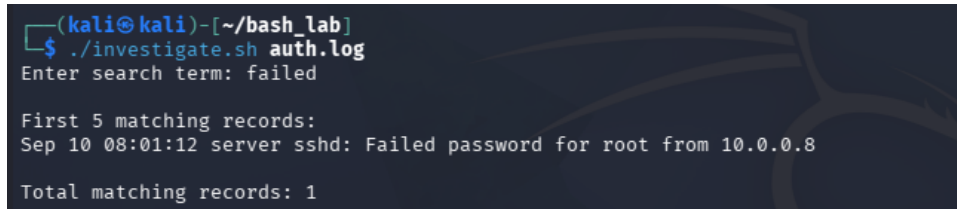
if [ ! -e "$logfile" ]; then
    echo "Error: '$logfile' does not exist."
    exit 1
fi

if [ ! -r "$logfile" ]; then
    echo "Error: '$logfile' is not readable."
    exit 1
fi

read -p "Enter search term: " term

echo
echo "First 5 matching records:"
grep -i "$term" "$logfile" | head -n 5

total=$(grep -ic "$term" "$logfile")
echo
echo "Total matching records: $total"
```



```
(kali㉿kali)-[~/bash_lab]
$ ./investigate.sh auth.log
Enter search term: failed

First 5 matching records:
Sep 10 08:01:12 server sshd: Failed password for root from 10.0.0.8

Total matching records: 1
```

Figure 13: investigate.sh prompting for a search term and reporting matches for 'failed'.

Observation / Explanation: Once the file passes both checks, read -p asks me for a search term. I typed 'failed' and it found the one matching line through grep -i piped into head -n 5. It also runs grep -ic separately just to get the total count (1) that way the count isn't limited by the head -n 5 cutoff.



```
(kali㉿kali)-[~/bash_lab]
$ ./investigate.sh nofile.log
Error: 'nofile.log' does not exist.
```

Figure 14: investigate.sh run against a log file that does not exist.

Observation / Explanation: nofile.log doesn't exist, so it fails the -e check right away and exits before it even asks me for a search term.

7. Summary: File Test Operators Used

Operator	Tests for	Used in
-e	Path exists (any type)	check_file.sh, filetype_checker.sh, check_empty.sh, investigate.sh
-f	Regular file	filetype_checker.sh, log_search.sh, check_empty.sh
-d	Directory	filetype_checker.sh
-r	Read permission	filetype_checker.sh, investigate.sh
-w	Write permission	filetype_checker.sh
-x	Execute permission	filetype_checker.sh
-s	Exists and size > 0 (non-empty)	check_empty.sh

8. Conclusion

Doing this lab, I noticed all five scripts basically follow the same pattern: check the argument count first, check the file's existence/type with the right operator, and only then actually do something with the file. Skipping any of those checks is usually what causes a script to crash with a confusing error instead of a clean message. The log_search.sh and investigate.sh scripts were the most useful ones combining grep, head, and -c for counting turned a few simple permission checks into something that actually works like a basic log-analysis tool.